# Current State Analysis

## 1. What exists today

The current application is already a workable non-AI prototype.

- `dev_test/py/non_ai_flow/app.py` boots a `FastAPI` app and wires in the GitHub router, dashboard UI, and Jenkins service.
- `dev_test/py/non_ai_flow/jenkins_service.py` is the main orchestration module today.
- `dev_test/py/non_ai_flow/github_service.py` already contains reusable GitHub fetch and single-file branch push helpers.
- `dev_test/py/non_ai_flow/dashboard_ui.py` provides operator/tester dashboards for webhook testing, inspection, and manual triggering of fix steps.
- `dev_test/py/non_ai_flow/runtime/requests.jsonl` acts as the current persistence layer for captured webhook events.

## 2. Current request flow

Current runtime flow:

1. Jenkins posts to `POST /api/v1/webhooks/jenkins/failure`.
2. The request body is stored as a `StoredEvent`.
3. The service extracts `console_text` from the payload.
4. The service immediately runs `_analyze_event(event)`.
5. The analysis result is stored back into the event record.
6. Separate endpoints can prepare a fix proposal and then push a branch / create a PR.

## 3. Current strengths

There is more reusable functionality here than a blank-slate design would suggest.

- Webhook ingestion already exists and returns `202`.
- Event persistence already exists and includes payload, summary, console text, and analysis/proposal slots.
- The GitHub service can already fetch dependency-related files from a repo branch.
- The GitHub service can already create or reuse a branch and update a target file.
- The current code already understands the distinction between general fixes and BOM-friendly fixes at a very basic heuristic level.
- `pyproject.toml` already includes `langgraph`, `langchain`, and `langchain-openai`, so the dependency groundwork is present even though LangGraph is not yet used in the runtime code.

# 4. Current architectural limits

## 4.1 The analyzer is single-pattern and shallow

`_analyze_event()` in `jenkins_service.py` assumes it can identify one package, one current version, one recommended version, and one target file from console text.

That assumption breaks on real Trivy-style scan output where:

- many vulnerabilities appear in one payload
- one package can have several CVEs
- one finding can list several fixed versions
- remediation may require parent/BOM/property upgrades instead of direct dependency version replacement

## 4.2 The real payload already demonstrates the gap

The stored sample in `dev_test/py/non_ai_flow/runtime/requests.jsonl` contains a large Trivy dependency scan with many CVEs across `pom.xml`, but the saved analysis result is:

- `supported = false`
- `issue_type = "UNSUPPORTED"`
- `target_file = "pom.xml"`
- many CVEs collected, but no structured issue list

This is the strongest proof point for the new LangGraph design: the webhook capture is working, but the reasoning layer is not yet able to transform the captured failure into an actionable multi-step remediation plan.

## 4.3 BOM awareness is only heuristic right now

The current code has helpful intent:

- `_solution_kind_for_target()` distinguishes BOM-friendly dependency upgrades from Docker/package upgrades.
- `_patch_pom_xml()` tries direct dependency replacement first and then property replacement.

But it does not actually reason about:

- parent POM upgrades
- imported BOMs
- dependencyManagement
- Gradle `platform(...)`, `enforcedPlatform(...)`, or version catalogs
- multi-module repos
- one upgrade fixing many CVEs at once

## 4.4 GitHub write support is single-file oriented

The current push helper updates one target file per call.

That is enough for simple demos, but the target LangGraph flow will often need one or more of:

- `pom.xml` plus `Dockerfile`
- root `build.gradle` plus module `build.gradle`
- `gradle.properties` plus `settings.gradle`
- multiple manifests changed as part of a single BOM-friendly fix

Node 4 therefore needs multi-file commit support rather than the current single-file content update path.

## 4.5 Orchestration is synchronous inside the webhook path

Today the webhook endpoint stores the event and analyzes it in the same request lifecycle. That is acceptable for regex parsing, but it is not a good fit for:

- LangGraph execution

- LLM calls
- larger repo file fetches
- validation and retry logic

The target design should keep the webhook fast and use background graph execution after persistence.

# 5. Reusable modules for the new design

The proposed LangGraph implementation should reuse the following instead of replacing them:

- FastAPI app setup in `dev_test/py/non_ai_flow/app.py`
- webhook route shape and event persistence ideas from `dev_test/py/non_ai_flow/jenkins_service.py`
- GitHub file retrieval from `dev_test/py/non_ai_flow/github_service.py`
- branch naming conventions and compare/PR concepts from the current GitHub push flow
- dashboard patterns if you later want an AI-run monitor page

# 6. Recommended design direction

Recommended transition:

1. Keep current non-AI endpoints working.
2. Add a new AI orchestration layer beside them.
3. Move issue extraction, fix planning, and validation into LangGraph nodes.
4. Reuse GitHub service helpers where possible, but introduce a multi-file write abstraction for Node 4.

# 7. Key design decisions

- Webhook remains the ingress boundary.
- LangGraph becomes the reasoning/orchestration boundary.
- GitHub service becomes the repository I/O boundary.
- BOM validation becomes an explicit node rather than an implicit regex or patching side effect.

- Branch push must happen only after the validation node approves the change plan.